Generative Modeling, Probabilistic Models, Variational Inference, and Variational Autoencoders

Rishant Mallick

August 2026

Contents

1 Introduction

This report develops a systematic view of probabilistic and generative modeling, beginning with the distinction between generative and discriminative models and progressing to directed graphical models, maximum likelihood, latent-variable models, variational inference, and the Variational Autoencoder (VAE). The organization follows the concepts and terminology in the supplied study material. The central theme is that a generative model attempts to represent the distribution and mechanism underlying observations, whereas a discriminative model focuses directly on the prediction relationship needed for a task.

2 Generative and Discriminative Modeling

2.1 Generative modeling

A generative model attempts to model how observations could have been generated. Depending on the formulation, it may model a joint distribution such as

$$p(x, y), \tag{1}$$

or a conditional/marginal distribution such as

$$p(x \mid y), \qquad p(x). \tag{2}$$

The conceptual question is:

> How could the observed data have been generated?

2.2 Discriminative modeling

A discriminative model directly learns the relationship required for prediction. For classification, the probabilistic form is

$$p(y \mid x), \tag{3}$$

while a deterministic view is simply

$$x \longrightarrow y. \tag{4}$$

The conceptual question is:

> Given the observation, what should be predicted?

2.3 Conceptual direction

The arrows below are conceptual rather than a requirement that every model implement a deterministic mapping:

$$\text{Generative:} \qquad y \longrightarrow x, \tag{5}$$

$$\text{Discriminative:} \qquad x \longrightarrow y. \tag{6}$$

A generative model may instead explicitly model $p(x, y)$ or $p(x \mid y)$.

2.4 Generative modeling and scientific modeling

Generative modeling is attractive when the underlying mechanism matters. A conceptual physical process can be represented as

$$\text{underlying factors/process} \longrightarrow \text{observable phenomenon}. \tag{7}$$

For example, physical variables such as temperature, pressure, wind, and humidity can jointly determine observable weather. Similarly, a model of an image may conceptually involve object identity, three-dimensional structure, lighting, camera parameters, projection, and finally pixels.

2.5 Physical constraints and causality

A generative formulation can incorporate known physical constraints together with unknown factors or noise:

$$\text{known structure} + \text{unknown factors/noise} \longrightarrow \text{observations}. \tag{8}$$

This can provide an interpretable representation of the assumed data-generating mechanism. The supplied material also emphasizes that modeling a generating process can expose causal relationships, which can be more transferable across environments than purely correlational patterns.

2.6 Using a generative model for discrimination

Suppose A and B are two possible classes and x is observed data. A generative model may provide

$$p(x \mid A), \qquad p(x \mid B). \tag{9}$$

Bayes' rule gives the desired posterior classification probability:

$$p(A \mid x) = \frac{p(x \mid A)p(A)}{p(x)}. \tag{10}$$

Thus, generative information can be converted into discriminative information.

2.7 Why not always use a generative model?

Learning a complete high-dimensional distribution can be computationally expensive. If the sole objective is classification and sufficient labeled data are available, learning the entire data-generating process may be unnecessary. A direct discriminative model can focus on

$$x \longrightarrow y \tag{11}$$

or $p(y \mid x)$.

2.8 Bias, assumptions, and amount of data

Generative models often impose stronger assumptions about the data distribution. If those assumptions are incorrect, the model can introduce bias. The supplied material emphasizes the following practical trade-off:

- With limited labeled data, learning broader data structure can be useful.
- With abundant labeled data, a discriminative model may directly learn the task without modeling unnecessary details.
- With many unlabeled examples and few labeled examples, generative modeling can support semi-supervised learning by exploiting the structure of all available data.

3 Generative Modeling for Representation Learning

Generative modeling is not restricted to sample generation. It can serve as an auxiliary learning objective that encourages a model to learn useful internal representations.

For temporal data, predicting a future observation can force the representation to capture useful information about objects, motion, spatial relationships, and interactions:

$$x_t \longrightarrow x_{t+1} \quad \Longrightarrow \quad \text{useful representation}. \tag{12}$$

A general conceptual chain is

$$\text{generative/prediction task} \longrightarrow \text{representation} \longrightarrow \text{downstream task}. \tag{13}$$

Table 1: Conceptual comparison of generative and discriminative modeling.

Property	**Generative**	**Discriminative**
Primary goal	Model data generation/distribution	Direct prediction
Typical distribution	$p(x, y)$, $p(x \mid y)$, $p(x)$	$p(y \mid x)$
Conceptual direction	Factors/classes $\rightarrow$ data	Data $\rightarrow$ prediction
Advantages	Structure, constraints, limited-label settings	Direct task optimization
Disadvantages	Stronger assumptions and computational cost	May require substantial labeled data

3.1 Unsupervised representation learning

The goal is to learn useful internal representations without manually supplied labels. Desirable properties discussed in the supplied material include:

- **Disentanglement:** different latent dimensions represent different factors of variation.
- **Semantic meaning:** latent variables correspond to meaningful concepts.
- **Statistical separation:** different latent factors ideally contain separate information.
- **Causal relevance:** representations ideally reflect underlying causes rather than only correlations.

3.2 Unsupervised versus self-supervised learning

These terms should not be treated as identical. Unsupervised representation learning describes a *goal*: learn useful representations without human-provided labels. Self-supervised learning describes a *strategy*: construct a supervisory signal automatically from the data itself.

Examples include masked-token prediction, next-token prediction, masked-image reconstruction, and contrastive objectives. Thus,

$$\text{self-supervised learning} \quad \text{can be used to achieve} \quad \text{unsupervised representation learning.} \tag{14}$$

4 Probabilistic Models

4.1 Definition

A probabilistic model represents uncertainty using probability distributions. Rather than asserting a single deterministic outcome, it represents multiple possible outcomes and their probabilities.

4.2 Random variables

A model may contain continuous and discrete random variables. Examples of continuous variables include temperature, pixel intensity, and microphone amplitude; examples of discrete variables include class labels, word tokens, and counts.

4.3 Joint distribution

For variables X, Y, Z, the most complete probabilistic description is their joint distribution:

$$p(x, y, z). \tag{15}$$

The joint distribution specifies how the variables are related and forms the basis for deriving marginal and conditional distributions.

4.4 True distribution and model distribution

Let the unknown true data-generating distribution be denoted by

$$p^*(x). \tag{16}$$

Observed data are samples from this unknown distribution:

$$x \sim p^*(x). \tag{17}$$

A parameterized model instead represents

$$p_\theta(x), \tag{18}$$

where θ contains the model parameters. Learning seeks parameters such that

$$p_\theta(x) \approx p^*(x). \tag{19}$$

For neural networks, θ typically contains weights and biases.

4.5 Unconditional and conditional models

An unconditional model describes the overall data distribution:

$$p_\theta(x). \tag{20}$$

A conditional model describes an output conditioned on an input:

$$p_\theta(y \mid x). \tag{21}$$

For image classification, x may be an image and y a class label.

4.6 Categorical distributions and softmax

For K classes, a neural network can produce logits $a_1, \ldots, a_K$. Softmax converts them into valid probabilities:

$$p_i = \frac{\exp(a_i)}{\sum_{j=1}^{K} \exp(a_j)}, \qquad \sum_{i=1}^{K} p_i = 1. \tag{22}$$

The conditional model can then be written as

$$p_\theta(y \mid x) = \text{Categorical}(y; p), \tag{23}$$

where p is produced by the neural network.

4.7 Neural networks as distribution parameterizers

A neural network does not replace probability theory. Rather, it can parameterize a probability distribution:

$$x \longrightarrow \text{neural network} \longrightarrow \text{distribution parameters}. \tag{24}$$

For classification, the parameters are class probabilities. For a Gaussian model, they may be μ and σ.

5 Directed Graphical Models and Bayesian Networks

A directed graphical model represents random variables as nodes in a directed acyclic graph (DAG). The graph specifies conditional dependencies.

5.1 Factorization of the joint distribution

For variables $x_1, \ldots, x_M$ ordered according to the DAG,

$$\boxed{p_\theta(x_1, \ldots, x_M) = \prod_{i=1}^{M} p_\theta\big(x_i \mid \mathrm{Pa}(x_i)\big)} \tag{25}$$

where $\mathrm{Pa}(x_i)$ denotes the parents of x_i.

For example, if $A \to B$ and $A \to C$,

$$p(A, B, C) = p(A)p(B \mid A)p(C \mid A). \tag{26}$$

Root nodes have no parents and therefore have unconditional distributions. Non-root nodes have conditional distributions given their parents.

5.2 Neural parameterization

Traditionally, conditional distributions can be parameterized using relatively simple functions. Modern deep generative models use neural networks to produce distribution parameters:

$$p_\theta(x \mid \mathrm{Pa}(x)). \tag{27}$$

The neural network therefore implements a flexible mapping from parent variables to parameters of the conditional distribution.

6 Maximum Likelihood Learning

6.1 Fully observed data

Suppose the dataset is

$$D = \{x^{(1)}, x^{(2)}, \ldots, x^{(N)}\}, \tag{28}$$

and the examples are independently and identically distributed (i.i.d.). The dataset likelihood factorizes as

$$p_\theta(D) = \prod_{n=1}^{N} p_\theta(x^{(n)}). \tag{29}$$

6.2 Log-likelihood

Because products can be numerically and computationally inconvenient, we maximize the log-likelihood:

$$\log p_\theta(D) = \sum_{n=1}^{N} \log p_\theta(x^{(n)}). \tag{30}$$

The maximum-likelihood estimator is

$$\boxed{\theta^* = \arg\max_\theta \log p_\theta(D)}. \tag{31}$$

Maximizing the log-likelihood is equivalent to maximizing the likelihood because the logarithm is a strictly increasing function.

6.3 Gradient ascent

The parameters can be updated in the direction of the likelihood gradient:

$$\theta \leftarrow \theta + \eta \nabla_\theta \log p_\theta(D), \tag{32}$$

where η is the learning rate.

6.4 Mini-batch stochastic gradient descent

Using the entire dataset for every update can be expensive. Instead, a mini-batch $B \subset D$ can provide an approximate gradient:

$$g_B = \nabla_\theta \sum_{x \in B} \log p_\theta(x). \tag{33}$$

The resulting update is stochastic because the gradient depends on the randomly selected mini-batch. Under standard sampling assumptions, the mini-batch gradient provides an estimate of the full-data gradient.

7 MAP Estimation and Bayesian Inference

7.1 Maximum likelihood

Maximum likelihood chooses parameters that maximize the likelihood:

$$\theta_{\mathrm{ML}} = \arg\max_\theta p_\theta(D). \tag{34}$$

7.2 Maximum a posteriori estimation

If a prior $p(\theta)$ is placed over parameters, MAP estimation maximizes the posterior:

$$\theta_{\mathrm{MAP}} = \arg\max_\theta p(\theta \mid D) = \arg\max_\theta p(D \mid \theta) p(\theta). \tag{35}$$

Equivalently,

$$\theta_{\mathrm{MAP}} = \arg\max_\theta \left[\log p(D \mid \theta) + \log p(\theta)\right]. \tag{36}$$

7.3 Full Bayesian inference

Rather than selecting a single parameter value, Bayesian inference represents uncertainty about the parameters through a posterior distribution:

$$p(\theta \mid D). \tag{37}$$

8 Latent-Variable Models

8.1 Introducing hidden variables

A latent-variable model introduces a hidden variable z that is not directly observed. The joint distribution becomes

$$\boxed{p_\theta(x, z)}. \tag{38}$$

A common generative structure is

$$z \longrightarrow x, \tag{39}$$

with factorization

$$p_\theta(x, z) = p(z) p_\theta(x \mid z). \tag{40}$$

8.2 Marginal likelihood

Only x is observed, so the model likelihood requires marginalization over the latent variable:

$$\boxed{p_\theta(x) = \int p_\theta(x, z)\, dz} \tag{41}$$

for continuous z, or

$$p_\theta(x) = \sum_z p_\theta(x, z) \tag{42}$$

for discrete z.

8.3 Why latent variables are useful

Latent variables can represent underlying factors that explain observations. Instead of modeling a complicated distribution directly in observation space, the model can describe a simpler latent space and then generate observations from it.

9 Deep Latent Variable Models

A deep latent variable model (DLVM) uses neural networks inside a latent-variable probabilistic model. The neural network can represent highly nonlinear relationships while probability distributions retain an explicit representation of uncertainty.

The model can therefore combine

$$\boxed{\text{probabilistic graphical model} + \text{deep neural network}}. \tag{43}$$

This combination is central to the VAE framework.

10 Variational Autoencoder

10.1 Generative structure

The simplest VAE generative model contains a latent variable z and an observation x:

$$z \sim p(z), \qquad x \sim p_\theta(x \mid z). \tag{44}$$

Thus,

$$\boxed{p_\theta(x, z) = p(z)p_\theta(x \mid z)}. \tag{45}$$

10.2 Prior

A common latent prior is the standard multivariate Gaussian:

$$\boxed{p(z) = \mathcal{N}(z; 0, I)}. \tag{46}$$

This prior provides a structured reference distribution for the latent space.

10.3 Decoder / generative model

The decoder represents the conditional likelihood

$$p_\theta(x \mid z). \tag{47}$$

The decoder neural network takes z and produces the parameters of the observation distribution. Therefore,

$$z \longrightarrow p_\theta(x \mid z) \longrightarrow x. \tag{48}$$

10.4 Encoder / recognition model

The encoder approximates the posterior distribution over latent variables:

$$\boxed{q_\phi(z \mid x) \approx p_\theta(z \mid x)}. \tag{49}$$

The encoder is a neural network with parameters ϕ. It maps an observation to parameters of an approximate posterior, commonly

$$\mu_\phi(x), \qquad \sigma_\phi(x), \tag{50}$$

with

$$q_\phi(z \mid x) = \mathcal{N}\left(z; \mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x))\right). \tag{51}$$

10.5 Encoder and decoder as coupled components

The VAE has two neural networks with different parameters:

$$\text{Encoder:} \quad x \longrightarrow q_\phi(z \mid x), \tag{52}$$

$$\text{Decoder:} \quad z \longrightarrow p_\theta(x \mid z). \tag{53}$$

They are separate networks, but training couples them: the encoder must produce a latent representation from which the decoder can reconstruct the observation.

10.6 Representation learning interpretation

The decoder imposes a useful constraint on the encoder. If z contains insufficient information about x, the decoder cannot reconstruct the observation accurately. Hence the reconstruction requirement encourages the latent variable to preserve useful information.

11 Bayesian Interpretation of the Encoder

By Bayes' rule,

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)}. \tag{54}$$

The denominator is

$$p_\theta(x) = \int p_\theta(x \mid z)p(z)\, dz, \tag{55}$$

which can be computationally difficult for expressive models. Consequently, the VAE uses $q_\phi(z \mid x)$ as a tractable approximation to the true posterior.

12 Variational Inference and Amortized Inference

12.1 Why ordinary variational inference can be expensive

For each observation x_i, one could introduce a separate variational distribution $q_i(z)$. For a dataset with a very large number of observations, optimizing a separate distribution for every example is inefficient.

12.2 Amortized inference

A VAE instead learns a single inference function:

$$\boxed{q_\phi(z \mid x)}. \tag{56}$$

The parameters ϕ are shared across all observations. Different inputs produce different posterior parameters, even though the encoder parameters are the same:

$$x_1 \to (\mu_1, \sigma_1), \qquad x_2 \to (\mu_2, \sigma_2), \qquad x_3 \to (\mu_3, \sigma_3). \tag{57}$$

Thus, amortized inference means that the model learns a reusable inference function rather than a separate optimized inference solution for each example.

12.3 Inference for a new observation

After training, a new observation can be processed by a forward pass:

$$x_{\text{new}} \longrightarrow \text{encoder} \longrightarrow q_\phi(z \mid x_{\text{new}}). \tag{58}$$

This is the key computational advantage of amortization.

13 Reparameterization Trick

13.1 The sampling problem

The encoder defines a distribution from which a latent sample is required:

$$z \sim \mathcal{N}(\mu, \sigma^2). \tag{59}$$

Direct random sampling makes ordinary backpropagation through the sampling operation difficult.

13.2 Reparameterized sampling

The sampling operation is rewritten as

$$\boxed{z = \mu + \sigma\epsilon}, \tag{60}$$

where

$$\epsilon \sim \mathcal{N}(0, 1). \tag{61}$$

For the neural network,

$$z = \mu_\phi(x) + \sigma_\phi(x)\epsilon. \tag{62}$$

The randomness is isolated in ϵ, while $\mu_\phi(x)$ and $\sigma_\phi(x)$ remain differentiable functions of the network parameters. This allows gradient-based optimization through the latent sampling computation.

14 VAE Objective

The VAE balances two goals: reconstruction and regularization of the latent distribution.

14.1 Reconstruction

The decoder should assign high probability to the observed data under the inferred latent variable. Conceptually,

$$x \approx \hat{x}. \tag{63}$$

14.2 Latent regularization

The approximate posterior is encouraged to remain close to the prior, commonly $\mathcal{N}(0, I)$. This is represented by a KL-divergence term.

14.3 Evidence lower bound

For a single observation, the standard VAE evidence lower bound (ELBO) is

$$\mathcal{L}_{\mathrm{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)} \left[\log p_\theta(x \mid z)\right] - D_{\mathrm{KL}} \left(q_\phi(z \mid x) \,\|\, p(z)\right). \tag{64}$$

The first term encourages reconstruction; the second regularizes the latent posterior toward the prior. Maximizing the ELBO provides a tractable variational objective related to maximum likelihood.

15 Binary Image Example

Suppose an image contains binary pixels:

$$x_j \in \{0, 1\}. \tag{65}$$

The latent variable can still use a Gaussian prior:

$$p(z) = \mathcal{N}(z; 0, I). \tag{66}$$

The decoder neural network takes z and produces a probability vector

$$p = \mathrm{Decoder}_\theta(z), \tag{67}$$

where each component satisfies

$$0 \le p_j \le 1. \tag{68}$$

The observation model is Bernoulli:

$$p_\theta(x_j \mid z) = \mathrm{Bernoulli}(x_j; p_j). \tag{69}$$

15.1 Why sigmoid is used

For a decoder logit a_j, the sigmoid function

$$\sigma(a_j) = \frac{1}{1 + e^{-a_j}} \tag{70}$$

produces a valid Bernoulli probability:

$$p_j = \sigma(a_j) \in (0, 1). \tag{71}$$

Thus, the decoder output is a probability for each binary pixel, not itself a Gaussian random variable.

15.2 Bernoulli log-likelihood

For one binary pixel,

$$p(x_j \mid z) = p_j^{x_j} (1 - p_j)^{1 - x_j}. \tag{72}$$

Taking the logarithm gives

$$\log p(x_j \mid z) = x_j \log p_j + (1 - x_j) \log(1 - p_j). \tag{73}$$

Assuming conditional independence of pixels given z,

$$\boxed{\log p(x \mid z) = \sum_j \left[x_j \log p_j + (1 - x_j) \log(1 - p_j)\right]}. \tag{74}$$

16 Why Latent-Variable Maximum Likelihood Becomes Intractable

16.1 Marginal likelihood

For a latent-variable model,

$$p_\theta(x) = \int p_\theta(x, z)\, dz = \int p(z) p_\theta(x \mid z)\, dz. \tag{75}$$

Although the joint distribution $p_\theta(x, z)$ can be easy to evaluate, the integral over a high-dimensional continuous latent space may be computationally intractable.

16.2 Effect on maximum likelihood

Maximum likelihood requires maximizing

$$\log p_\theta(x). \tag{76}$$

If $p_\theta(x)$ cannot be evaluated efficiently, direct maximum-likelihood optimization becomes difficult.

16.3 The posterior is also difficult

The posterior is

$$\boxed{p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}}. \tag{77}$$

Because both the numerator and the marginal likelihood involve the latent-variable model, exact posterior inference is generally difficult.

16.4 Key distinction

The important computational distinction is therefore:

- The joint distribution $p_\theta(x, z)$ can be tractable to evaluate.
- The marginal likelihood $p_\theta(x)$ may require an intractable integral.
- The posterior $p_\theta(z \mid x)$ may consequently also be intractable.

This motivates variational inference and the VAE encoder.

17 Connection to Expectation-Maximization

EM is a classical approach for latent-variable models. It alternates between estimating hidden variables and updating model parameters.

17.1 E-step

The E-step estimates the posterior or responsibilities of the latent variables. For example,

$$p(z_i \mid x_i, \theta). \tag{78}$$

17.2 M-step

The M-step uses the inferred latent-variable information to update the generative-model parameters θ.

17.3 Relation to the VAE

The VAE does not simply run classical EM separately for every observation. Instead, it learns a recognition model

$$x \longrightarrow q_\phi(z \mid x), \tag{79}$$

which provides a fast approximation to the posterior. This is amortized variational inference. The conceptual connection is therefore:

$$\text{infer latent variables} \longrightarrow \text{use them to learn generative parameters.} \tag{80}$$

18 VAE versus GAN

A Generative Adversarial Network (GAN) and a VAE are both generative frameworks, but they emphasize different properties.

18.1 VAE

The VAE is a latent-variable probabilistic model based on an explicit likelihood formulation and variational inference. It provides an explicit latent representation and is naturally suited to density modeling and representation learning. Generated samples can, however, be less sharp in some image-generation settings.

18.2 GAN

A GAN contains a generator and discriminator. The generator maps latent noise to samples, while the discriminator attempts to distinguish generated samples from real samples. GANs can produce highly sharp and perceptually realistic samples, but training can be difficult and the generator can fail to represent some regions of the data distribution (mode collapse/incomplete support).

Table 2: Conceptual comparison of VAE and GAN.

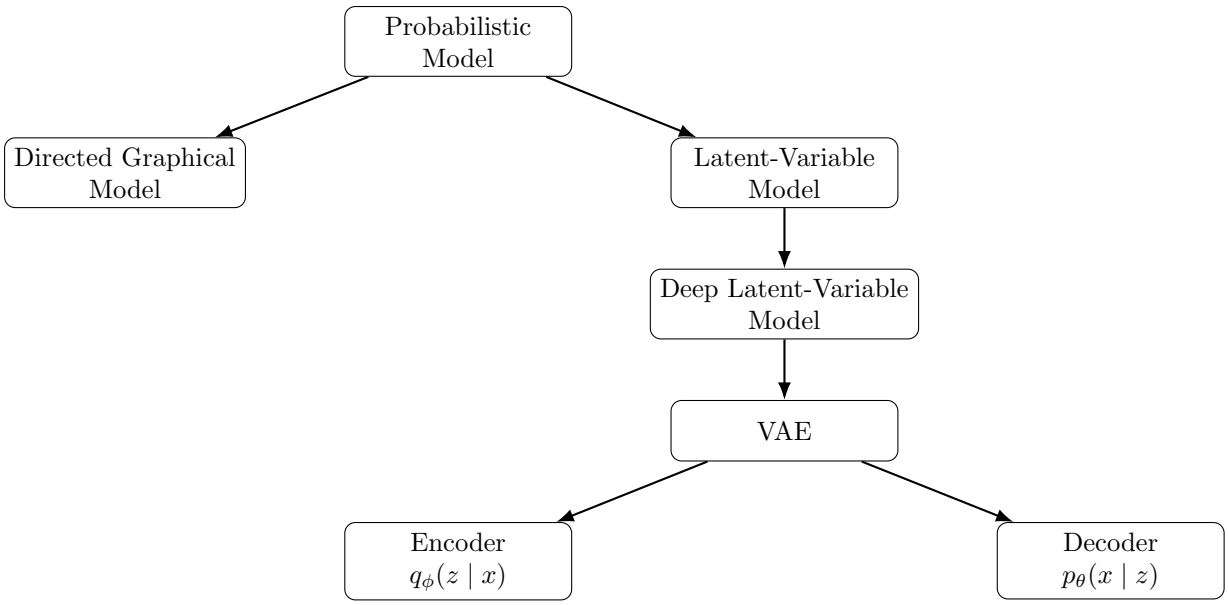
Property	**VAE**	**GAN**
Core mechanism	Latent-variable probabilistic model	Generator–discriminator game
Likelihood	Explicit likelihood framework	Not directly likelihood-based
Latent representation	Explicit	Usually implicit
Sample quality	Can be smoother/blurry	Often very sharp
Distribution coverage	Generally encourages broad coverage	Can miss modes
Training	Variational objective	Adversarial optimization
Representation learning	Strong natural connection	Not necessarily the main objective

19 Historical Connection: Helmholtz Machine

The supplied material connects VAEs historically to the Helmholtz Machine, which also combined a generative model with a recognition model. The Helmholtz Machine used the wake-sleep algorithm. The VAE instead derives its learning objective from a variational lower bound related to maximum likelihood, providing a unified objective for learning the generative and recognition networks.

20 Complete Conceptual Architecture

The major relationships can be summarized as follows:



21 Key Equations

$$\text{Generative joint model:} \quad p_\theta(x, z) = p(z)p_\theta(x \mid z), \tag{81}$$

$$\text{Marginal likelihood:} \quad p_\theta(x) = \int p_\theta(x, z)\, dz, \tag{82}$$

$$\text{Posterior:} \quad p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}, \tag{83}$$

$$\text{Variational posterior:} \quad q_\phi(z \mid x) \approx p_\theta(z \mid x), \tag{84}$$

$$\text{Reparameterization:} \quad z = \mu_\phi(x) + \sigma_\phi(x)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{85}$$

$$\text{ELBO:} \quad \mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - D_{\text{KL}}(q_\phi(z \mid x) \| p(z)), \tag{86}$$

$$\text{Maximum likelihood:} \quad \theta^* = \arg\max_\theta \log p_\theta(D). \tag{87}$$

22 Mental Model for the Entire Topic

The concepts can be organized into the following chain:

$$\boxed{\begin{array}{c}\text{Real-world process}\\ \downarrow\\ \text{Probability distribution}\\ \downarrow\\ \text{Graphical model / generative model}\\ \downarrow\\ \text{Latent variables, if useful}\\ \downarrow\\ \text{Deep latent-variable model}\\ \downarrow\\ \text{VAE: variational inference + neural networks}\\ \downarrow\\ \text{Learned latent representation + generation}\end{array}} \tag{88}$$

The essential distinction to remember is:

$$\boxed{\text{Generative modeling: model the data-generating distribution/process}} \tag{89}$$

whereas

$$\boxed{\text{Discriminative modeling: directly model the prediction relationship}} \tag{90}$$

For a VAE specifically,

$$\boxed{x \xrightarrow{q_\phi(z|x)} z \xrightarrow{p_\theta(x|z)} x} \tag{91}$$

with the latent prior

$$p(z) = \mathcal{N}(0, I), \tag{92}$$

and the reparameterized latent sample

$$z = \mu_\phi(x) + \sigma_\phi(x)\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{93}$$

23 Conclusion

Generative modeling provides a probabilistic description of how observations arise, while discriminative modeling directly targets prediction. Directed graphical models express probabilistic dependencies through factorization of a joint distribution. Maximum likelihood learns parameters from observed data, while latent-variable models introduce hidden variables that can provide a compact description of underlying factors.

The VAE combines these ideas with deep neural networks. Its decoder defines a generative model $p_\theta(x \mid z)$, its encoder defines an amortized variational approximation $q_\phi(z \mid x)$, and the reparameterization trick makes stochastic latent sampling compatible with gradient-based learning. The resulting framework simultaneously supports probabilistic generation and structured representation learning.